```
build:
  - 'composer install --no-dev'
```

Troubleshooting

While leveraging Vapor to drive your Laravel applications can significantly enhance load speeds, it's important to address potential performance issues stemming from misconfigured infrastructure.

This section delves into a series of key techniques for optimizing Vapor infrastructure, with the aim of bolstering the speed of your Laravel applications. Before we delve into these techniques, it's crucial to consider the following points:

- **Prioritize application optimization:** Boosting your application's speed might eliminate the need for any modifications to the infrastructure. Identify and address performance bottlenecks within the application by implementing fundamental strategies such as caching, queuing, or optimizing database queries.

Be mindful of the potential impact on your AWS expenses: Implementing changes to your Vapor environment by adding or adjusting resources could lead to an increase in your AWS bill.

- **Utilizing fixed-capacity databases:**

Serverless databases are designed to scale automatically, without a predefined limit on RAM or disk space. However, AWS's method of *auto-scaling* can lead to brief delays known as cold starts whenever additional database resources are needed within your environment.

To mitigate this challenge, consider implementing *fixed-capacity* databases that come with pre-allocated resources. You can easily create these databases using either the Vapor UI or the dedicated database CLI command.

- **Opting API Gateway v2 over API Gateway v1 for improved performance:**

Transitioning from API Gateway v1 to API Gateway v2 can lead to a significant 50% decrease in query latency for your application. However, it's important to note that API Gateway v2 operates on a regional basis, potentially impacting clients located in distant regions. Furthermore, certain